

EC327 — Week 1 Preview

Chapters 1-3 from a textbook in progress

Jeffrey Carruthers

May 2026

Table of contents

Preface	1
Who this book is for	1
How the book is organized	1
Conventions used in this book	2
On using AI while reading this book	2
Acknowledgments	2
I Week 1 — From Python to C++	3
1 Course intro, first C++ file, Linux basics	7
1.1 What this course is	7
1.2 Lab 0: the ungraded standup	7
1.3 Anatomy of a C++ file	8
1.4 The compile / link / run cycle	9
1.5 A first program you can extend	10
1.6 Linux basics, in 90 seconds	10
1.7 What to do before the next session	11
1.8 What you should understand by the end of this chapter	11
2 Types, operators, selection, and the compile cycle	13
2.1 The plan: quadratic0.cpp through quadratic3.cpp	13
2.2 Stage 0: variables, input, and output	14
2.3 Stage 1: arithmetic and <code><cmath></code>	15
2.3.1 What goes wrong	17
2.4 Stage 2: selection — the <code>if</code> statement	17
2.5 Stage 3: three-way selection and the <code>and</code> keyword	18
2.6 A second worked example: a grade calculator	20
2.6.1 Stage 1: a sequence of independent <code>ifs</code>	21
2.6.2 Stage 2: a single <code>if</code> / <code>else if</code> / <code>else chain</code>	21
2.6.3 Where the progression goes next	22
2.7 The compile/link cycle, revisited	23
2.8 Interpreted vs compiled	23
2.9 Whitespace is not significant	24
2.10 Practice: ways to break each stage	25
2.11 What you should understand by the end of this chapter	25
3 <code>std::vector</code>, <code>range-for</code>, and your first <code>struct</code>	27
3.1 Stage 3: parallel arrays and the <code>while</code> loop	27
3.2 Why we do not write C-style arrays in 2026	29

Table of contents

3.3	Stage 4: <code>std::vector</code> replaces the C-style array	29
3.4	Iterating a vector — three idioms you should know	30
3.4.1	Idiom 1: range-based <code>for</code> (<i>the default you reach for first</i>)	31
3.4.2	Idiom 2: indexed <code>for</code> (<i>when the position matters</i>)	31
3.4.3	Idiom 3: iterator-based (<i>seen everywhere, written rarely</i>)	31
3.5	Your first <code>struct</code> : putting fields together	32
3.6	Stage 5: <code>vector<Grade></code> + <code>range-for</code> + early <code>break</code>	33
3.7	Structured bindings: peeking inside a row	34
3.8	Two-dimensional structures	35
3.8.1	Option 1: <code>vector<vector<T>></code> (<i>the flexible default</i>)	35
3.8.2	Option 2: <code>array<array<T, N>, M></code> (<i>when the size is fixed and small</i>)	35
3.8.3	Option 3: flat row-major (<i>when performance is the brief</i>)	35
3.9	What changes in stages 6 through 9 (a preview)	36
3.10	Practice: try to break each stage	37
3.11	What you should understand by the end of this chapter	37

Preface

This book is the companion text for **EC327, Introduction to Software Engineering**, taught in the first six-week summer session at Boston University by Jeffrey Carruthers. It is built directly from the lectures, labs, and in-class demos of the course, and it tracks the course’s day-by-day arc one chapter at a time.

Who this book is for

You have written code before — most likely in **Python**, possibly in MATLAB or JavaScript, occasionally in something compiled. You know how to write a function, a conditional, and a loop, and you have probably encountered objects. You have not necessarily written C++, you have not necessarily compiled anything from a shell, and you may not yet have a clear mental model of what happens between “I saved a file” and “the program ran.”

This book takes that as its starting line. It is not a *first* book about programming. It is, instead, a book about **what changes** when you move from a managed, interpreted, garbage-collected language into a compiled language that exposes the machine — and about the engineering discipline that makes such code livable.

How the book is organized

The chapters are grouped into six parts, one per week of the course. Each chapter corresponds to one lecture session (L1, L2, ..., L23). The opening chapters take you from a Python mindset into a working C++ inner loop; the middle chapters introduce objects, the layered machine, pointers, and modern memory management; the later chapters reach a deliberate climax with linked structures and a small binary search tree implemented from first principles, then pivot to generics, complexity, tooling, code review, and the final capstone project.

A separate **Lab Tooling** appendix collects the build-system, sanitizer, debugger, version-control, and unit-testing material that comes up across the labs but does not belong inside the conceptual flow.

Conventions used in this book

- **Code listings** are formatted to the course's `.clang-format` style. Where the course's style differs from common practice, the choice is named.
- **Callouts** flag specific kinds of advice:
 - *Note* — clarifying detail
 - *Tip* — a practical move that experienced C++ programmers reach for
 - *Warning* — a pitfall worth remembering
 - *AI touchpoint* — a moment where the course explicitly asks you to bring an AI assistant into the loop
- **and / or / not** are this book's primary spellings for the logical operators, with `&&` / `||` / `!` named on first use and used freely in any imported third-party code. You must be able to read both.

On using AI while reading this book

This course does not treat AI assistants as a contamination risk to be policed. It treats them as a normal part of the modern programming environment, and asks you to use them deliberately. Throughout the book, **AI touchpoints** mark moments where an LLM is the right next tool — explaining a compiler error, generating a unit test, reviewing a function, translating an idiom from Python into C++. You are expected to disclose AI use on graded work; the eight named roles, the disclosure form, and the discipline around what is still cheating even with this stance are spelled out in the AI Tools chapter.

Acknowledgments

Material is reused, adapted, and modernized from the author's previous EC327 offerings between 2018 and 2025. Earlier versions of several chapters appeared as instructor notes in the Summer 2022, Summer 2023, and Summer 2024 sections.

— J. Carruthers, Boston, 2026

Part I

Week 1 — From Python to C++

The opening week takes you from a working Python developer to a working C++ developer with a compiled, running first program, a configured toolchain, and the beginnings of a development rhythm. By the end of these three sessions every reader should have Multipass (or WSL2) running, have compiled and run a program, have used `clang-format` at least once, and have typed `gdb` at least to inspect a variable.

The chapters of this part:

- Chapter 1 — course intro, anatomy of a C++ file, Linux basics, first program
- Chapter 2 — types, expressions, the compile/link cycle, the `and/or/not` keywords, `clang-format` on save
- Chapter 3 — `std::vector<T>`, range-based for, structured bindings, the first user-defined `struct`

1 Course intro, first C++ file, Linux basics

The first day of EC327 has four jobs. By the time you leave you should know **what this course is asking of you**, you should have a **working development environment** you can return to tomorrow, you should be able to **read a small C++ file aloud** and explain what each line does, and you should have a small, working mental model of what *a Linux shell* is.

This chapter walks each in turn.

1.1 What this course is

EC327 is the second programming course in the BU ECE undergraduate sequence. It is a course in **software engineering** taught through the vehicle of **modern C++** (C++20, in 2026). The choice of C++ matters: C++ is the language in which most performance-critical and most embedded code is written, and it *exposes the machine* in ways that Python deliberately hides. The course's deeper goal is for you to leave with a mental model of *what is actually happening* when a program runs, and with the engineering discipline — testing, debugging, version control, code review — that lets you ship code you trust.

You will not be required to buy a textbook. (You have one here, but it is free.) You are *recommended* to read Stroustrup's *Programming: Principles and Practice Using C++* alongside this book; it is the canonical C++-from-scratch text by the language's designer.

1.2 Lab 0: the ungraded standup

The course's first interaction with a terminal is **Lab 0**, an ungraded walkthrough we do together in class. Its goal is to get every machine in the room — Mac, Windows, Linux — to a common standing-start state:

1. **Pick a terminal.** macOS: the built-in `Terminal.app` or `iTerm2`. Windows: Windows Terminal with WSL2 underneath.
2. **Install Multipass** (Mac) or **WSL2 + Ubuntu** (Windows). Both give you a real Linux environment beneath your everyday OS. Native Linux users already have what they need.

3. **Lay out two directories:** `~/ec327/course/` for read-only material the instructors hand you, and `~/ec327/work/` for everything *you* write. You will be told once, and reminded often: **never edit inside course/**. Copy a file out, then edit the copy.
4. **Clone the course repository** into `course/`.
5. **Install the toolchain:** `g++` (or `clang++`), `clang-format`, `clang-tidy`, `gdb`, `git`.
6. **Pick an editor.** Sublime, VS Code, Neovim, and Helix all have working C++ LSP setups; per-editor instructions live in the course's *Editor Setup* handout (in the Tools & Development Environment section of the course site).
7. **Compile and run the starter program.** See Section 1.5 below.

We do all of this *together*, on a projected screen, with everyone in the room typing along. If your machine breaks during the walkthrough, that is the best possible time for it to break.

Security through-line: the *blast radius* of `sudo`

Lab 0 is also the first appearance of the course's *security through-line*. Whenever you type `sudo` you are handing a piece of code permission to do anything on your machine. Pause for a second before each `sudo` and answer: *what code, exactly, am I about to run as root?* This is the same discipline that Claude Code's permission prompts try to teach — the prompt is not in your way, it is asking the question you should have asked yourself.

1.3 Anatomy of a C++ file

Here is the smallest interesting C++ program. Open it line by line.

```
// hello.cpp - your first program in EC327, Summer 2026.

#include <iostream>
#include <string>

int main() {
    std::string name = "EC327";
    std::cout << "Hello, " << name << "! This is C++.\n";
    return 0;
}
```

There are five things to notice, in order.

The comment. Lines starting with `//` are *line comments*; everything from `//` to the end of the line is for humans, ignored by the compiler. A file should always carry one identifying comment at the top. There is also a `/* ... */` block-comment form. You will use line comments 99% of the time.

The two `#include` lines. These are *preprocessor directives*. They are not C++ statements; they are textual instructions to a separate program (the *preprocessor*) that runs before the C++ compiler proper. `#include <iostream>` brings in the declarations needed for `std::cout`; `#include <string>` brings in the declarations needed for `std::string`. In Python you write `import math` and call `math.sqrt`; in C++ you `#include <cmath>` and call `std::sqrt`. The mechanics are different — Python *imports* a module object at runtime, C++ *splices in* a header file at compile time — but the mental model (“I am pulling in code someone else wrote”) is the same.

The `std::` prefix. Names in the C++ standard library live inside a *namespace* called `std`, and `::` is the operator that says “look inside that namespace.” So `std::cout` is the `cout` that lives inside `std`. This is direct analogue of Python’s `math.sqrt`. You will see code (often older code) that says `using namespace std`; near the top so that `cout` works without the prefix. **This course does not write `using namespace std`; in any code we distribute** — there are good reasons, which we will come to in the Modular Design chapter. For now: every standard library name is `std::` prefixed.

`int main()`. This is the *entry point* of the program. When you run a compiled C++ program, the operating system finds a function called `main` and calls it. The `int` says `main` returns an integer; that integer is the program’s *exit code* (0 means success, anything else means failure).

The body, in braces. The block `{ ... }` is what `main` actually does: declare a string, write a line to standard output, return 0. Statements end in semicolons. Whitespace and newlines inside a block have no meaning to the compiler — they exist for *you*.

1.4 The compile / link / run cycle

In Python you run a `.py` file directly:

```
$ python hello.py
```

In C++ the lifecycle has two distinct steps, and you need a working mental model of both.

```
$ g++ -std=c++20 -Wall -Wextra -o hello hello.cpp    # compile + link
$ ./hello                                           # run
Hello, EC327! This is C++.
```

The first command does three things at once:

1. **Preprocess** — the preprocessor handles every `#include`, splicing the header files into the source, and processes any `#define` or conditional compilation directives. The output is a single, large *translation unit*.
2. **Compile** — the compiler translates the translation unit into *machine code* for your specific CPU, packaged in an *object file* (a `.o` file you do not normally see when you use `-o`).

3. **Link** — the linker stitches the object files together with the standard library and produces an executable file named `hello`.

The executable is yours, runnable directly, no interpreter needed. This is the biggest difference between your C++ inner loop and your Python inner loop: you *compile*, you *run*, and if either step fails you fix something and try again.

i The course’s compile invocation

For this course we compile with `-std=c++20 -Wall -Wextra` as a baseline. `-std=c++20` selects the C++20 language standard; `-Wall -Wextra` asks the compiler to warn about a wide range of suspect-looking code. In Lab 2 we add `-Wpedantic` and a small set of sanitizers. The full course `Makefile` and a working `CMakeLists.txt` are in the Build Systems appendix.

1.5 A first program you can extend

In class we extend `hello.cpp` by hand, one small step at a time. The goal is that you finish the first session having *compiled at least four versions* of this same program — not because you needed four versions, but because we are deliberately training the **compile-every-few-lines, run-every-compile** inner loop that is your single most important habit this semester.

The progression looks roughly like this. Each version compiles and runs before moving to the next:

1. Print “Hello, EC327!” — the listing above.
2. Read a name from `std::cin` and greet it back. (Hello to your `<iostream>` input side: `std::cin >> name;`.)
3. Greet it three times by hand, with three `std::cout` lines. (We are *deliberately* not writing a loop yet — that comes tomorrow.)
4. Greet it once, but with a *starter-code* `Point` struct already defined for you in `point.h`, used to print the cursor position. (Your first exposure to a header you did not write; your first use of a struct.)

Each step is its own little change. Each ends with `g++` and `./hello`.

1.6 Linux basics, in 90 seconds

You do not need to be a Linux expert to take this course. You need a working mental model of three things.

The shell vs the terminal. A *terminal* is a window that types characters to a program and shows you what that program types back. A *shell* (`bash`, `zsh`, ...) is a particular program — the one that reads what you type, finds the program you named, and runs it. Today: open your terminal

and notice the *prompt* (typically ending in `$`). Type a command and press enter. That is the shell handing you a *new* prompt after the command finished.

Files and directories. The shell always has a *current directory* (`pwd` prints it). You move with `cd`, list with `ls`, look at file contents with `cat`, edit with your editor of choice. The full reference card is the course's *Terminal Cheatsheet* handout (in the Tools & Development Environment section of the course site).

Processes and `argv`. When you ran `./hello`, the shell *forked* a new process, *exec'd* your program into it, waited for it to finish, and printed the next prompt. Your program runs as its own *process* with its own memory. If you pass arguments — `./hello Alice` — the shell hands them to your program through `argv`, which we will use in earnest in the File I/O chapter.

1.7 What to do before the next session

1. Finish Lab 0 if anything did not work in class. (Office hours are open.)
2. Re-type `hello.cpp` from scratch, without copy/paste. Compile and run it.
3. Modify it three different ways and compile each time.
4. Read Chapter 2.

1.8 What you should understand by the end of this chapter

- What a `.cpp` file looks like and why each part is there.
- What `#include` does and how it compares to Python's `import`.
- What the `std::` prefix means.
- The three steps of the compile/link/run cycle.
- The shape of your development environment for the semester.
- The course's *compile every few lines, run every compile* rhythm.

If any of these feel shaky, you are not behind — you are on schedule. We will revisit each in the next two sessions.

2 Types, operators, selection, and the compile cycle

The first chapter got you a running program. This chapter introduces the small set of language pieces you need to write programs that actually *compute* something interesting: **variables and types**, **operators**, **input from the user**, and **selection** (the `if` statement and friends). The running example is a program almost every engineering student already understands as mathematics — the **quadratic formula** — built up in four stages, each adding exactly one new language feature.

This chapter also takes the second look at the **compile/link cycle** you saw at the end of Chapter 1, this time with enough vocabulary to talk about *why* it works the way it does.

2.1 The plan: quadratic0.cpp through quadratic3.cpp

The four stages are deliberately small. Each one compiles. Each one runs. Each one is broken in a way the next one fixes.

Stage	What it does	What's new
0	reads <code>a</code> , <code>b</code> , <code>c</code> and prints them back; hard-codes “roots are 3 and 5”	variables, <code>std::cin</code> , <code>std::cout</code>
1	computes the real roots using <code>std::sqrt</code> ; breaks if discriminant is negative	<code><cmath></code> , arithmetic operators
2	detects the complex case and refuses to compute it	comparison (<code><</code>), <code>if</code> -statement, exit codes
3	handles real-distinct, real-repeated, complex, and <code>a==0</code> linear cases	<code>if/else if/else</code> , <code>and</code> keyword

The full source for all four lives in `ec327/textbook/code/quadratic{0,1,2,3}.cpp`. You should plan to type each one in by hand (no copy-paste) the first time through. The point of this chapter is the *typing*, not the reading.

2.2 Stage 0: variables, input, and output

```
// quadratic0.cpp
#include <iostream>

int main() {
    double a{};
    double b{};
    double c{};

    std::cout << "Please enter the coefficients a, b, c: ";
    std::cin >> a >> b >> c;

    std::cout << "You typed " << a << " " << b << " " << c << ".\n";
    std::cout << "(The roots are 3 and 5. Probably not, but we will fix that.)\n";

    return 0;
}
```

There are five things to notice.

Declarations with a type. `double a{};` says “make a variable named `a`, its type is `double` (a floating-point number with about 15 digits of precision), and initialize it to zero.” In Python you would just write `a = 0.0` and the language would *infer* a type. In C++ the type comes *before* the name, and you have to write it. This is *static typing* — the compiler knows the type of every variable before the program runs, and uses that to catch entire classes of bugs at compile time.

Uniform initialization with `{}`. The empty braces, `double a{};`, mean “default-initialize”. For `double` that means zero. You can also write `double a{3.14};` to initialize to a particular value. The `{}` form works uniformly for every type — `int`, `double`, `std::string`, your own classes, arrays — and it is the form this book uses by default. The older forms `double a = 0.0;` and `double a(0.0);` mean the same thing here and you will see them in older code.

Uninitialized variables are a real bug class

If you write `double a;` (no `{}`) the variable has *some* value — whatever bits were already in that piece of stack memory. Reading it before assigning to it is *undefined behavior*: the program might appear to work, might crash, might give a different answer every run. The course’s `.clang-tidy` enables the `cppcoreguidelines-init-variables` check, which flags this at lint time. Default-initializing with `{}` is the modern habit that makes the warning go away and the code correct.

Reading input with `std::cin`. The `>>` operator on `std::cin` reads one value at a time, skipping whitespace. `std::cin >> a >> b >> c;` reads three values in a row. If the user types `1 -5 6` and hits enter, `a` is 1, `b` is -5, `c` is 6. If the user types 1 and hits enter, the program waits for the next number. Python’s analogue is `input()` plus a manual parse; C++’s `>>` parses while it reads.

Writing output with `std::cout`. The `<<` operator on `std::cout` writes one value at a time, in left-to-right order. You can chain as many as you like. The `"\n"` at the end of a string literal is a newline character; without it, output keeps going on the same line. (You may have seen `std::endl` in older code; we prefer `"\n"` because `std::endl` also forces a *flush* of the output buffer, which is rarely what you want.)

return 0;. The integer that `main` returns is the program’s *exit code*. By convention, 0 means “everything went fine”. Other values mean “something went wrong, and here’s a number telling you what.” We use this in stage 2.

Compile and run

```
$ g++ -std=c++20 -Wall -Wextra -o quadratic0 quadratic0.cpp
$ echo "1 -5 6" | ./quadratic0
Please enter the coefficients a, b, c: You typed 1 -5 6.
(The roots are 3 and 5. Probably not, but we will fix that.)
```

The `echo ... |` is a *pipe* — the shell hands the string `"1 -5 6"` to your program’s standard input as if you had typed it. Useful for non-interactive testing.

2.3 Stage 1: arithmetic and <cmath>

The mathematics: the roots of $ax^2 + bx + c = 0$ are

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

The C++:

```
// quadratic1.cpp -- new bits only
#include <iostream>
#include <cmath>

int main() {
    double a{}; double b{}; double c{};
    std::cout << "Please enter the coefficients a, b, c: ";
```

```

std::cin >> a >> b >> c;

double disc = b * b - 4 * a * c;

double root1 = (-b + std::sqrt(disc)) / (2 * a);
double root2 = (-b - std::sqrt(disc)) / (2 * a);

std::cout << "The roots are " << root1 << " and " << root2 << ".\n";
return 0;
}

```

Three things changed.

#include <cmath> and std::sqrt. The C++ standard library spreads its mathematics functions across a small set of headers. `<cmath>` gives you `std::sqrt`, `std::pow`, `std::sin`, `std::log`, `std::abs`, and friends. Notice that `std::sqrt` carries the `std::` prefix for the same reason `std::cout` did in Chapter 1: it lives in the `std` namespace.

Arithmetic operators. The five standard ones, with their meaning:

Operator	Meaning	Note
+	addition	
-	subtraction (binary) and negation (unary)	-b is unary - applied to b
*	multiplication	
/	division	integer if both operands are integer; floating if either is floating
%	remainder (modulo)	integer operands only

A subtlety worth remembering: in C++, `13 / 5` is 2, not 2.6. Both operands are integers, so integer division is used. To get 2.6 you write `13.0 / 5` or `13 / 5.0`. (Python 3 made the opposite choice — `13 / 5` is always 2.6, and integer division gets the special `//` operator.)

Precedence and parentheses. The expression `b * b - 4 * a * c` evaluates in the order you would expect from mathematics: the multiplications happen first, *then* the subtraction. C++ has a *precedence table* that pins down the order for every operator combination. You do not need to memorize it. You *do* need to use parentheses freely whenever the order matters and is not obvious

— every senior C++ programmer reads $(a + b) * c$ faster than $a + b * c$, even though only the first one *requires* parentheses.

i The full operator-precedence table

You can find the complete operator-precedence table at https://en.cppreference.com/w/cpp/language/operator_precedence. It is worth glancing at once and then bookmarking. We will add operators to it throughout the book.

The intermediate variable for `disc`. Notice that $b * b - 4 * a * c$ appears in both root expressions. We compute it once, store it in `disc`, and use `disc` in both places. This is the same DRY principle (“*Don’t Repeat Yourself*”) you learned in Python, and the same reason: if the formula has a typo, you want it to have the typo once, not twice.

2.3.1 What goes wrong

Try running stage 1 with 1 2 5:

```
$ echo "1 2 5" | ./quadratic1
```

Please enter the coefficients a, b, c: The roots are nan and nan.

`nan` stands for *not a number* — `std::sqrt(-16)` returned the IEEE-754 sentinel value for an undefined operation, and we then propagated it through the arithmetic. The program ran to completion, but the answer is wrong. Stage 2 makes it notice.

2.4 Stage 2: selection — the `if` statement

```
// quadratic2.cpp -- new bits only
double disc = b * b - 4 * a * c;

if (disc < 0) {
    std::cout << "Sorry, the roots are complex. Try stage 3 of this program.\n";
    return 1;
}

double root1 = (-b + std::sqrt(disc)) / (2 * a);
double root2 = (-b - std::sqrt(disc)) / (2 * a);
```

Two things changed.

The if statement. The shape is `if (condition) { body }`. The *condition* is an expression that evaluates to either `true` or `false`. If it's `true`, the body runs; if `false`, the body is skipped and execution continues after the closing brace.

In `if (disc < 0)`, the condition is the expression `disc < 0`. The `<` operator compares two numbers and produces a `bool` — either `true` or `false`. C++ has six comparison operators:

Operator	Meaning
<code>==</code>	equal to
<code>!=</code>	not equal to
<code><</code>	less than
<code><=</code>	less than or equal to
<code>></code>	greater than
<code>>=</code>	greater than or equal to

 `=` is assignment; `==` is comparison

`if (a = 0) { ... }` is a *bug*, not a syntax error. It assigns 0 to `a` (throwing away whatever `a` had) and then tests whether the value of that assignment, which is 0, is “truthy” — it isn't, so the body never runs. `g++` with `-Wall` flags this with **suggest parentheses around assignment used as truth value**. Heed the warning. We will revisit at length in the Common Errors appendix.

The non-zero return. Stage 2 returns 1 (not 0) in the failure case. A non-zero exit code is the conventional way for a Unix program to tell its caller that something went wrong. You can inspect the exit code of the last program you ran with `echo $?`:

```
$ echo "1 2 5" | ./quadratic2
Please enter the coefficients a, b, c: Sorry, the roots are complex. Try stage 3 of this pro
$ echo $?
1
```

The shell, makefile rules, CI systems, and other programs that compose yours all look at the exit code to decide whether to keep going.

2.5 Stage 3: three-way selection and the `and` keyword

```
// quadratic3.cpp -- the interesting bits only
if (a == 0 and b == 0) {
    std::cout << "a and b are both zero; this is not a quadratic.\n";
    return 1;
}
```

```

}

if (a == 0) {
    // Degenerate linear case:  $bx + c = 0 \Rightarrow x = -c/b$ .
    std::cout << "Linear (a is zero). The root is " << -c / b << ".\n";
    return 0;
}

double disc = b * b - 4 * a * c;

if (disc > 0) {
    double root1 = (-b + std::sqrt(disc)) / (2 * a);
    double root2 = (-b - std::sqrt(disc)) / (2 * a);
    std::cout << "Two real roots: " << root1 << " and " << root2 << ".\n";
} else if (disc == 0) {
    double root = -b / (2 * a);
    std::cout << "One repeated real root: " << root << ".\n";
} else {
    double real_part = -b / (2 * a);
    double imag_part = std::sqrt(-disc) / (2 * a);
    std::cout << "Two complex roots: "
                << real_part << " + " << imag_part << "i and "
                << real_part << " - " << imag_part << "i.\n";
}

```

This is the full program. Three more language pieces appear.

if / else if / else. C++ does not have a dedicated **elif** keyword the way Python does. Instead, **else if** is just an **if** placed inside the **else** branch of an earlier **if**, with the braces left off because the body is itself a single statement. The chain runs *top to bottom* and stops at the first branch whose condition is **true**. The final **else** (no condition) runs if none of the earlier branches matched. Three-way selection — *bigger / equal / smaller*, *positive / zero / negative*, *win / draw / lose* — is common enough that this idiom shows up constantly.

Logical operators with their keyword spellings. The condition `a == 0 and b == 0` uses the **and** keyword to combine two comparisons. C++ has had the spellings **and**, **or**, and **not** as full keywords since C++11, semantically identical to **&&**, **||**, and **!**:

Keyword	Symbolic	Meaning
and	&&	logical AND
or	 	logical OR
not	!	logical NOT

This book writes **and**, **or**, **not** as the **primary spelling** because for this course’s audience — students who already know Python — `if (finished or not found)` reads more transparently than `if (finished || !found)` does. Both forms compile to identical machine code. Neither is enforced or banned by the course’s linter. You will see the symbolic forms `&& || !` in *every* C++ library, AI-generated suggestion, Stack Overflow answer, and textbook other than this one — **you must remain fluent in reading both**.

Why this book made this choice

Two reasons. First, the visual one: `&&` and `&` differ by one character, in the same family — typing one when you meant the other is a real bug class (`if (a & b)` and `if (a && b)` both compile and both do something, but they do very different things). **and** and `&` are *not* visually confusable. Second, the pedagogical one: in a Python-expat audience the keyword forms read like a natural-language sentence on first encounter, which buys back some attention for the actual logic. Stroustrup himself uses the symbolic forms throughout *Programming: Principles and Practice*; we are deliberately diverging on this one point.

Compound statements (blocks). A pair of braces `{ ... }` groups a sequence of statements into a single *compound statement* (a “block”). Any single statement can be replaced by a block. The body of an `if` is a single statement; if you want two statements there, wrap them in braces. This is *how* the chain `if (...) { ... } else if (...) { ... } else { ... }` works mechanically: each `{ ... }` is a single compound statement playing the role of `if`’s body.

Always brace your `if` bodies

You *can* leave the braces off a single-statement body: `if (a < 0) return 1;` is legal. The course style is to always brace anyway:

```
if (a < 0) {
    return 1;
}
```

The reason is one of the most expensive bugs in C history — the Apple SSL “*goto fail*” bug of 2014, where a misindented unbraced `if` body silently turned off all certificate verification on iPhones for months. Always-brace is the discipline that prevents whole categories of edit-time mistakes. `clang-format` will adjust the spacing for you; it cannot insert the braces you forgot to type.

2.6 A second worked example: a grade calculator

The quadratic series is the right *first* exercise because it forces you to confront `<cmath>`, comparison, and a three-way branch in a context that is unmistakably mathematics. But selection shows up

everywhere in real code, not just in the math library. The second worked example takes a problem familiar to every undergraduate — *turning a numerical score into a letter grade* — and walks through the same selection ideas in a fresh setting.

The full progression of this example has nine stages (`grade_v1.cpp` through `grade_v9.cpp`); each later stage replaces the selection logic with a more sophisticated data-structure technique. The later stages need ideas from Chapter 3 and beyond, so we stop at stage 2 here. The progression picks up again in chapter 3.

2.6.1 Stage 1: a sequence of independent ifs

```
// grade_v1.cpp -- the interesting part only
if (g < 50.0) { let = 'F'; }
if (g >= 50.0 and g < 60.0) { let = 'D'; }
if (g >= 60.0 and g < 70.0) { let = 'C'; }
if (g >= 70.0 and g < 80.0) { let = 'B'; }
if (g > 80.0) { let = 'A'; }    // BUG: should be g >= 80.0
```

Read this carefully before reading the rest of this paragraph. *What is the shape of the bug?* Try the program on input 80:

```
$ echo 80 | ./grade_v1
Enter the student's grade: The letter grade is  .
```

There is a blank where the letter should be. The student got a perfect score of 80.0. None of the five conditions matched (80 < 50 no, 50 ≤ 80 < 60 no, ..., 80 > 80 no), so `let` was never assigned. We *did* default-init `let` to `'\0'` with the empty `{}`, so the program prints what looks like a space — undefined-behavior-by-strict-reading was avoided by the default-initialization, but the *output* is still wrong.

There is also a more insidious bug in this version that the input 80 does not expose: every condition is tested independently. If two conditions both match (because we wrote the boundaries inconsistently), the *later* one wins. We do not have that problem here, but a future maintainer who adds a new range without thinking carefully *will*.

2.6.2 Stage 2: a single if / else if / else chain

```
// grade_v2.cpp -- the interesting part only
if (g < 50.0) {
    let = 'F';
} else if (g < 60.0) {
```

```
    let = 'D';
} else if (g < 70.0) {
    let = 'C';
} else if (g < 80.0) {
    let = 'B';
} else {
    let = 'A';
}
```

Two things happened.

The boundary bug is gone. Every branch except the last uses a strict `<`. A score of exactly 79.999 is a B; a score of exactly 80.0 falls through to the final `else` and is an A. We did not even need to write `>= 80.0` anywhere — the unconditional `else` says “*everything that didn’t match above*”. That is a real engineering virtue: the partitioning logic is expressed *once*, in one place, and is exhaustive by construction.

The “let was never assigned” failure mode is gone. The chain is exhaustive — for any double value of `g`, exactly one branch runs. `let` is always set.

```
$ echo 80 | ./grade_v2
Enter the student's grade: The letter grade is A.
```

The pattern, named

Whenever you find yourself writing several `ifs` in a row to handle disjoint cases of the same value, the rewrite to `if / else if / else` is almost always the right move. The cases become *visibly* mutually exclusive, the partitioning is forced to be exhaustive (or you write a final `else` that explicitly handles the “I forgot a case” path), and a future reader does not have to mentally re-check whether any two conditions could overlap.

2.6.3 Where the progression goes next

`grade_v3.cpp` notices that the `if/else if/else` chain is itself a repeated pattern (compare-against-threshold, assign-corresponding-letter) and replaces it with two parallel arrays and a `while` loop. `grade_v4.cpp` swaps the C-style arrays for `std::vector`. `grade_v5.cpp` introduces a `struct Grade` and replaces the parallel-vectors-with-an-index pattern with a single `vector<Grade>` and a range-based `for`. The arc continues through `grade_v9.cpp`, picking up early-break, enums, and a small refactor into a free function. **We do not need any of that yet** — it is the job of Chapter 3 and the Functions chapter to introduce those pieces. The point here is that you can already see the *shape* of the refactor: a problem that looked like five separate branches turns out to be a *table*,

and once it is a table the language gives you better ways to express it. That is one of the recurring rhythms of the book.

2.7 The compile/link cycle, revisited

Look again at the command from Chapter 1:

```
$ g++ -std=c++20 -Wall -Wextra -o quadratic3 quadratic3.cpp
$ ./quadratic3
```

You now have the vocabulary to read it more carefully.

1. **Preprocess.** Every `#include` line is a directive to the *preprocessor* to splice the contents of that header into the file. After preprocessing, `quadratic3.cpp` is no longer 50 lines; it is tens of thousands of lines, because `<iostream>` and `<cmath>` brought in everything they declare. The output is a single *translation unit*.
2. **Compile.** The compiler reads the translation unit, type-checks it (catching, for example, `std::sqrt("hello")` before the program ever runs), and translates it into machine code for your CPU. The output is an *object file*.
3. **Link.** The linker stitches your object file together with the standard library's pre-compiled object files (so `std::cout` and `std::sqrt` resolve to real code) and produces the *executable*.

C++ does all three in this one `g++` command. They are conceptually distinct and you will eventually want to invoke them separately (e.g. to compile two `.cpp` files into one program — see the Functions chapter and the Build Systems appendix).

Why this matters, and what it's costing you. Each step adds *time*. For a 50-line program this is unnoticeable; for a 50,000-line project it can be minutes. But for a 50-line program you also get something Python cannot give: a guarantee, before the program ever runs, that every type matches up, every name resolves, every function you called exists and has the right signature. The compiler is your first and most relentless reviewer. **Listen to its warnings.** `-Wall -Wextra` is the bare minimum; we will add more in the Build Systems appendix.

2.8 Interpreted vs compiled

Why does C++ do it this way and Python does not? The short answer is *when the work happens*.

- **Python** ships your `.py` file to an *interpreter* (the `python` program). The interpreter reads each line as the program runs, figures out at that moment what type each variable has, looks up each method by name in a dictionary, and dispatches. Almost all of the work is done *at run time*.

2 Types, operators, selection, and the compile cycle

- C++ does most of the work *at compile time*. The compiler resolves types, selects the right overload of `std::sqrt`, lays out the memory for each variable, and emits CPU instructions that the CPU executes directly. At run time there is no interpreter, no dictionary lookup per call, no type check.

Each choice has tradeoffs. Python is faster to *write* and slower to *run*; C++ is the reverse. Python lets you change a function's signature in one place and try it immediately; C++ asks you to recompile every file that depended on that signature, but in exchange catches the call sites you forgot to update.

For the kind of code most software engineers ship — a programs that runs on a million phones, a service that handles a thousand requests per second, a chunk of firmware that boots an aircraft — that tradeoff is overwhelmingly in C++'s favor on the *run* side, and the engineering practices in the rest of this book are largely about making the *write* side bearable enough that the tradeoff is worth it.

2.9 Whitespace is not significant

If you came from Python, this is the most jarring difference in the language.

In Python, indentation *defines* a block. In C++, indentation is purely a visual hint for humans; *braces* define the block, and *semicolons* terminate statements. All four of these compile to the same program:

```
if (x < 0) { std::cout << "negative\n"; }

if (x < 0) {
    std::cout << "negative\n";
}

if    (x<0)    {std::cout<<"negative\n";}

if
(
    x
    <
    0
)
{ std::cout << "negative\n"; }
```

This is a freedom that gets abused. The course response is: **clang-format on save**. The starter repo ships a `.clang-format` config (Google baseline, two-space indent, 80-column lines, attached

braces, left-align `*` and `&`). Configure your editor to run it on save and the question of “where does the brace go” becomes uninteresting. The course will not award or deduct points for formatting; the linter does that mechanically.

We will revisit `clang-format` in the Sanitizers appendix when we wire it into the build.

2.10 Practice: ways to break each stage

The point of having four stages is that *each one is broken*. Before moving on, spend ten minutes trying to break them on purpose. For each stage, ask:

1. What input produces wrong output without an obvious error?
2. What input crashes the program (or makes it loop forever)?
3. What input does the program *refuse* to handle, with a clear message?

Stage 0 is broken because it lies (it always says “3 and 5”). Stage 1 is broken because it returns `nan` silently for the complex case. Stage 2 is broken because it returns an error message instead of the complex roots that do exist. Stage 3 is *less* broken — it handles every input we have discussed — but it has a subtle bug: `if (a == 0)` is comparing a floating-point number to exactly zero. For an input like `0.00000001 1 1`, stage 3 thinks `a` is not zero and solves the quadratic correctly; for `0.0 1 1` it solves the linear case correctly; for something between, the answer can be numerically terrible. We do not fix this here. It is a real bug, and one of the standing themes of the book: every program has bugs, and the discipline is *knowing which bugs you decided not to fix*.

AI touchpoint — explain a compiler error

Open `quadratic1.cpp` and delete the `#include <cmath>` line. Compile it. You will get an error something like `'sqrt' is not a member of 'std'`. Paste the entire error message into an AI assistant and ask: “*What does this error mean and how do I fix it?*” Read the answer. Then ask: “*What other common errors look like this one?*” This is the **AI as IT aide** role from the AI Tools chapter, and it is the most useful single habit you can pick up in week 1.

2.11 What you should understand by the end of this chapter

- Declaring a typed variable and initializing it with `{}`
- The five arithmetic operators, integer vs. floating division, and when to use parentheses
- Reading input with `std::cin >>` and writing output with `std::cout <<`
- The six comparison operators and the difference between `=` and `==`
- The three logical operators in both their keyword (`and/or/not`) and symbolic (`&&/||/!`) forms
- Selection with `if`, `if/else`, and `if/else if/else`
- Why blocks are always braced in this course

2 Types, operators, selection, and the compile cycle

- The three steps of the compile/link cycle and why C++ does it this way
- That whitespace is not significant and `clang-format` settles the question

If you can read all four stages of the quadratic solver out loud and explain what each line is doing, you are ready for Chapter 3.

3 `std::vector`, `range-for`, and your first struct

Chapter 2 left you at a place where you can write programs that handle *one* value at a time. Read a number; classify it; print a letter. The world that matters to engineers is full of *collections* of values — students in a class, samples in a signal, pixels in an image, points along a trajectory — and the language tools for working with collections are different enough from the single-value tools that they deserve their own chapter.

This chapter introduces three intertwined ideas:

1. **`std::vector<T>`** — C++’s default sequence container, the workhorse you will reach for in 80% of programs you write this term.
2. **Iteration**, in the three idioms you should know on sight: the range-based `for`, the indexed `for`, and a brief look at iterators.
3. **`struct`** — your first user-defined type, the language piece that lets you say “*these N pieces of data go together*” once and treat the group as a single named thing.

The running example is the **grade-calculator progression you started in Section 2.6**. We pick it up at stage 3 and walk through stage 5. Each stage introduces exactly one new idea, and each stage’s last bug is what motivates the next stage.

3.1 Stage 3: parallel arrays and the `while` loop

`grade_v2` ended with a five-branch `if / else if / else` chain. The *structure* of those five branches is identical — compare `g` against a threshold, assign the corresponding letter. When the structure is identical the code is asking to be replaced by *data* (the table of thresholds and letters) plus a *loop* that walks the data.

```
// grade_v3.cpp -- the interesting bits only
double thresholds[5] = {50.0, 60.0, 70.0, 80.0, 100.0};
char letters[5]      = {'F', 'D', 'C', 'B', 'A'};

double g{};
std::cin >> g;

int i = 0;
```

3 `std::vector`, range-for, and your first `struct`

```
while (g >= thresholds[i]) {  
    i++;  
}  
char let = letters[i];
```

Three things appear for the first time.

C-style arrays. `double thresholds[5] = { ... };` declares a *fixed-size* array of five doubles. The `[5]` is the *length* (not a maximum — the length is part of the type), and the values inside the braces are the elements in order. You access an element by *subscripting* with `[i]`, where `i` is an integer index in the range `0..4` (zero-based, like Python).

The while loop. `while (condition) { body }` runs `body` repeatedly *while* `condition` is true. Check the condition; if true, run the body; check the condition again; if true again, run the body again; eventually the condition becomes false and the loop exits. The condition `g >= thresholds[i]` says “as long as my grade is still at or above the threshold at position `i`, move on to the next position.”

The index variable pattern. `int i = 0;` outside the loop, `i++` inside. This is the canonical *visit every position from 0 upward* idiom; we will see a tighter spelling of it (`for`) in a moment.

What goes wrong at the boundary

Run `grade_v3` with input 100:

```
$ echo 100 | ./grade_v3  
Enter the student's grade: The letter grade is .
```

Garbage in the output. Trace it: `i` starts at 0. `100 >= 50` is true, so `i` becomes 1. `100 >= 60` is true, so `i` becomes 2. `100 >= 70`, true, `i = 3`. `100 >= 80`, true, `i = 4`. `100 >= 100`, true, `i = 5`. Now the loop body has set `i = 5`, the condition re-checks `g >= thresholds[5]`, and we read the *sixth* element of a five-element array. There is no sixth element. The C-style array does not check the bounds; the program just reads whatever bits sit beyond the end and uses *that* as the threshold and the letter.

This is *undefined behavior* — the most feared phrase in the C++ vocabulary. The program does not error out; it does not stop; it produces *some* answer. The answer is wrong. Worse, the answer may be different on a different machine, with a different compiler, at a different time of day, in a different build configuration. We will name this category of bug repeatedly throughout the book.

3.2 Why we do not write C-style arrays in 2026

C-style arrays predate C++ entirely — they are 1972 Kernighan-and-Ritchie C, unchanged in essentials for half a century. They have four properties that modern C++ deliberately works around:

1. **They do not carry their size with them.** `thresholds[5]` is a `double[5]` at the point of declaration, but the moment you pass it to a function it *decays* to a bare pointer that has forgotten how long it is. Bounds checking becomes the caller's manual problem.
2. **They have no bounds checking at the subscript site.** `thresholds[5]` compiles fine even though there is no element 5.
3. **They cannot grow.** A `double[5]` is always exactly 5 elements.
4. **They cannot easily be assigned, returned from a function, or stored in other containers.**

Modern C++ gives you two replacements:

- `std::array<T, N>` — fixed size like the C-style array, but carries its size, lives properly in the type system, and supports `.size()`, `.at()`, range-for, and friends.
- `std::vector<T>` — variable-size, grows as you push elements onto it, the default everyday container. *This is the one you reach for first.*

Stage 4 swaps the C-style arrays for `std::vector`.

3.3 Stage 4: `std::vector` replaces the C-style array

```
// grade_v4.cpp -- the interesting bits only
#include <vector>

std::vector<double> thresholds{50.0, 60.0, 70.0, 80.0, 100.0};
std::vector<char> letters {'F', 'D', 'C', 'B', 'A'};

double g{};
std::cin >> g;

std::size_t i = 0;
while (g >= thresholds[i]) {
    i++;
}
char let = letters[i];
```

The *logic* is unchanged. The container is different.

3 `std::vector`, range-for, and your first `struct`

`std::vector<T>` and the `<vector>` header. `std::vector` is in the standard library; you `#include <vector>` to get it. The `<T>` is a *template parameter*: `std::vector<double>` is “a vector of doubles”, `std::vector<char>` is “a vector of chars”, `std::vector<std::string>` is “a vector of strings”. We will say more about templates in the Templates chapter; for now, treat the `<T>` like the `[T]` in Python’s `list[int]` type hint.

Brace initialization. `std::vector<double> thresholds{50.0, 60.0, ...};` uses the same `{ ... }` uniform-initialization syntax you saw with `double a{};` in ch02. With braces full of values, it initializes the vector with those elements in order. The vector’s size is whatever you put inside the braces — five, in this case.

`std::size_t` instead of `int`. Container sizes and indices in C++ are *unsigned* — they cannot be negative. The standard type for them is `std::size_t`. Using `int` here would technically work but would trigger a compiler warning about sign-compare in `i < thresholds.size()`-style expressions you will write five seconds from now. Start using `std::size_t` for indices early; it is one less warning to wade through later.

 `vector::operator[]` is *also* unchecked

The 100-input bug from `grade_v3` is not gone. `std::vector::operator[]` — the `[i]` notation — does **not** bounds-check either. The reason is performance: `operator[]` is designed to compile down to a single memory load, the same as the C-style array does, so that a tight inner loop has no hidden cost. If you want bounds checking, use `thresholds.at(i)` instead — it throws `std::out_of_range` on a bad index. We will dig into the tradeoff in the Debugger chapter. For now: the 100-input bug from stage 3 is *still here*. Stage 5 fixes it properly by changing the loop shape so the out-of-bounds case never happens.

3.4 Iterating a vector — three idioms you should know

Before we get to stage 5, take a slight detour. Once you have a vector, the single most common thing you do with it is *visit every element*. C++ has several spellings of this; three of them you should recognize on sight.

The running example: a vector of ints, printed.

```
std::vector<int> v{4, 5, -4, 1, 2, 7, 8, -9, 42, 10};
```

3.4.1 Idiom 1: range-based for (*the default you reach for first*)

```
for (auto x : v) {
    std::cout << x << ' ';
}
```

Read it as English: “*for each x in v , do...*”. The variable x takes on each element of v in order. The type is `auto`, meaning *the compiler figures out from v what x ’s type should be* — here, `int`. You do not say `int`, `size_t`, or anything about the iteration mechanics; the loop is about what you do with each element, not about how you get there.

When you need to *modify* the elements as you go, write `auto&` (reference to the element). When you only need to read them and they are expensive to copy (like a `std::string`), write `const auto&`.

3.4.2 Idiom 2: indexed for (*when the position matters*)

```
for (std::size_t i = 0; i < v.size(); i++) {
    std::cout << "v[" << i << "] = " << v[i] << '\n';
}
```

This is the C-style `for` loop you have probably written in Java or JavaScript. It has three semicolon-separated pieces: an *initializer* (`std::size_t i = 0`), a *test* (`i < v.size()`), and a *step* (`i++`). Use this when the *index* itself is part of what you are printing or computing, or when you need to skip elements (`i += 2`).

`v.size()` returns the current number of elements (as a `std::size_t`).

3.4.3 Idiom 3: iterator-based (*seen everywhere, written rarely*)

```
for (auto it = v.begin(); it != v.end(); ++it) {
    std::cout << *it << ' ';
}
```

`v.begin()` returns an *iterator* pointing at the first element; `v.end()` returns an iterator pointing *one past* the last element. The `*it` syntax *dereferences* the iterator to give you the element it is currently pointing at. `++it` advances to the next position.

You will not write iterators by hand often — the range-based `for` is almost always shorter. But every STL algorithm in the STL Algorithms chapter takes a pair of iterators (`begin`, `end`), so you need to know what they are.

💡 Which idiom to use, by default

- **Range-based for** for “do something with each element”.
- **Indexed for** when the index is part of the computation.
- **Iterator-based** only when an STL algorithm gives you iterators back, or when you need fine-grained control (skip elements conditionally, iterate partway through). Otherwise: range-based.

3.5 Your first struct: putting fields together

In `grade_v4` we had two parallel vectors: a vector of thresholds and a vector of letters. The two vectors had to stay the same length and the same *order*. If anyone ever edited one without editing the other, the program breaks silently — the letter for B would suddenly point at the threshold for C. That kind of constraint, where two *logically connected* pieces of data are held apart by the language, is exactly what a `struct` exists to fix.

```
struct Grade {  
    double threshold;  
    char letter;  
};
```

`struct` is the keyword. `Grade` is the new *type name*. The body is a list of *member declarations* — fields, in Python terms. After this declaration, `Grade` is a usable type, just like `int` or `std::string`.

You construct a `Grade` value with brace initialization in field-declaration order:

```
Grade g1{50.0, 'F'};           // threshold 50.0, letter 'F'  
Grade g2{60.0, 'D'};
```

You access fields with the `.` operator:

```
std::cout << g1.letter << " starts at " << g1.threshold << "\n";
```

And — the punch line — you can put `Grades` inside a `std::vector`:

```
std::vector<Grade> scale{  
    {50.0, 'F'},  
    {60.0, 'D'},  
    {70.0, 'C'},  
    {80.0, 'B'},
```

```
{100.0, 'A'},
};
```

Now the threshold-and-letter pairing is enforced by the type system. There is no way to mis-align the two vectors because there is only one vector.

i `struct` vs. `class`

Both keywords define a new user-defined type. In C++ the only language-level difference between `struct` and `class` is the *default access* of members (`struct` defaults to public, `class` defaults to private). The conventional distinction is *cultural*:

- Use **`struct`** when the type is a *bag of fields* with no real invariants beyond “these belong together” — `Grade`, `Point`, `Color`, `Rect`.
- Use **`class`** when the type *enforces something* — `BankAccount` (whose balance must be consistent), `LinkedList` (whose nodes must be reachable from the head), `File` (whose handle must be closed).

Chapter 5 takes up `class` in earnest. For this chapter, every user-defined type we write is a `struct`.

3.6 Stage 5: `vector<Grade>` + `range-for` + `early break`

```
// grade_v5.cpp -- the interesting bits only
struct Grade {
    double threshold;
    char letter;
};

std::vector<Grade> scale{
    {50.0, 'F'}, {60.0, 'D'}, {70.0, 'C'}, {80.0, 'B'}, {100.0, 'A'},
};

double g{};
std::cin >> g;

char let = '?';
for (const auto& row : scale) {
    let = row.letter;
    if (g < row.threshold) {
```

3 `std::vector`, range-for, and your first `struct`

```
        break;
    }
}
```

This is the version that finally *looks like* what the problem is asking for. Read the loop aloud: “*for each row in the scale, tentatively pick the row’s letter; if my grade is below the row’s threshold, that’s the answer.*”

Three small ideas pulled together.

`const auto& row` — for each iteration, `row` is a *const reference* to the current `Grade` in the vector. **`const`** because we are reading the row, not modifying it. **`&`** because we want to refer to the actual element without copying it (a `Grade` is small here, but the habit is the one you want when the elements are big).

Tentative assignment + check. We assign `let` *before* checking whether to stop. So after the loop runs to completion — even if no threshold triggered the `break` — `let` is the letter from the *last* row, which is 'A'. The 100-bug from stages 3 and 4 is gone: when `g = 100`, we walk all five rows, and the last `let = 'A'` is the answer.

`break`. The `break` statement exits the *innermost enclosing loop* immediately. It is the right tool for “I have found the answer, stop looking.” Like every selection or jump statement, it should be used sparingly enough that a reader is not surprised by it; one `break` in a loop, used at the natural exit point, is fine.

```
$ echo 100 | ./grade_v5
Enter the student's grade: The letter grade is A.
$ echo 49 | ./grade_v5
Enter the student's grade: The letter grade is F.
$ echo 75 | ./grade_v5
Enter the student's grade: The letter grade is B.
```

3.7 Structured bindings: peeking inside a row

A modern C++ shorthand worth knowing now even though you will use it rarely in week 1: you can destructure a `struct` (or a `std::pair`, or a `std::tuple`, or a fixed-size array) into named variables at the point of access:

```
for (const auto& [thr, let] : scale) {
    std::cout << let << " starts at " << thr << '\n';
}
```

The `[thr, let]` says “this row has two fields; bind them locally to `thr` and `let`”. The names you choose are local to the loop body. This is the same destructuring idea as Python’s `for k, v in d.items():`. We will use it again in the File I/O chapter when we are reading two-field records from a file.

3.8 Two-dimensional structures

Real engineering data often does not fit in a single sequence. An image is a 2D grid of pixels; a chessboard is an 8×8 grid of squares; a matrix is a 2D grid of numbers. C++ gives you three ways to represent a 2D grid, and which you pick affects performance and clarity.

3.8.1 Option 1: `vector<vector<T>>` (*the flexible default*)

```
std::size_t rows = 4, cols = 5;
std::vector<std::vector<int>> grid(rows, std::vector<int>(cols, 0));
grid[2][3] = 42;
```

Rows can be different lengths if you want (a *ragged* array). Either dimension can be resized at runtime. Indexing is the natural `grid[r][c]`. Memory is allocated in pieces — one allocation for the outer vector of vectors, plus one per row — which is *fine* for most programs but not ideal for tight numerical inner loops.

3.8.2 Option 2: `array<array<T, N>, M>` (*when the size is fixed and small*)

```
#include <array>
std::array<std::array<int, 5>, 4> grid{}; // 4 rows, 5 cols, zero-init
```

Same `grid[r][c]` indexing. The dimensions are *part of the type*, so they are checked at compile time. Memory is one contiguous block; no allocation, no extra layer of indirection. This is what you want when the dimensions are known ahead of time and small (a 3D rotation matrix, a chessboard).

3.8.3 Option 3: flat row-major (*when performance is the brief*)

```
std::vector<int> grid(rows * cols, 0);
auto at = [&](std::size_t r, std::size_t c) -> int& {
    return grid[r * cols + c];
};
at(2, 3) = 42;
```

3 `std::vector`, range-for, and your first `struct`

One contiguous block of memory, addressed by hand. This is the layout that your CPU's cache prefetcher loves; large numerical libraries like Eigen and NumPy use it underneath. The cost is that your code now has to know about the *stride* (`cols`). The tiny `at` lambda hides that arithmetic at the call sites.

💡 Which 2D representation, by default

- `vector<vector<T>>` when the sizes are dynamic and you do not care about the last 20% of performance.
- `array<array<T, N>, M>` when the sizes are known at compile time and small.
- **Flat row-major** when an inner numerical loop is the hot path. We will revisit this in the Complexity chapter.

A worked example: `box_of_stars` from the priors. Two nested `for` loops, no 2D container needed because we never store the grid — we just print it.

```
// box_of_stars.cpp
#include <iostream>
int main() {
    int X{}, Y{};
    std::cin >> X >> Y;
    for (int i = 0; i < X; i++) {
        for (int j = 0; j < Y; j++) {
            std::cout << '*';
        }
        std::cout << '\n';
    }
    return 0;
}
```

Read it twice. The *outer* loop runs `X` times, once per row. Inside each row, the *inner* loop runs `Y` times, once per column, printing one `'*'` each time. After the inner loop finishes, the outer loop's body prints a newline and moves on to the next row. This is the canonical shape of a nested `for`; you will see it in every image-processing loop you ever write, even when the *grid* lives somewhere else than on the screen.

3.9 What changes in stages 6 through 9 (a preview)

The grade-calculator progression keeps going, but the next changes are about *organization*, not iteration:

- `grade_v6` reorganizes the loop condition (testing `>` against the row threshold rather than `<`).

- `grade_v7` factors the loop out into a free function `char get_grade(double g)` — covered in the Functions chapter.
- `grade_v8/v9` lift the `vector<Grade>` and the lookup logic into a small class, `GradeScheme` — covered in the two Classes chapters.

The recurring point is that the *physics* of the problem (turn a grade into a letter) has not changed since stage 1. Every iteration of the program moves the burden of *expressing* that physics from the *code* (handwritten branches) into the *data* (a table) and then into the *type system* (a struct, a class). That trajectory — push work from code into data into types — is one of the central rhythms of good engineering, and we will return to it.

3.10 Practice: try to break each stage

As with chapter 2, every stage of the progression is broken in a way the next stage fixes. For each of `grade_v3`, `v4`, `v5`, ask:

1. What input crashes it or produces wrong output?
2. What input would you *expect* to crash but does not?
3. Which fix does the next version apply, and would a *different* fix also work?

For example: `grade_v4` could be fixed *without* introducing a struct, by using `.at()` instead of `[]` — that would throw a clean exception at the 100-input case rather than silently reading garbage. Is that a better fix than what `v5` does? Why or why not?

💡 AI touchpoint — generate alternative implementations

Open `grade_v4.cpp` and ask an AI assistant: “Show me three different ways to handle the bounds-check problem on input 100, ranging from minimal to idiomatic.” Read the answers critically. Which would you ship? Which would you reject as over-engineering? This is the **AI as test-case generator + reviewer** role from the AI Tools chapter, applied to a tiny problem you already understand.

3.11 What you should understand by the end of this chapter

- Why we do not write C-style arrays in 2026 (and what we write instead)
- Declaring a `std::vector<T>` with brace initialization
- That `vector::operator[]` is unchecked and `.at()` is the safe alternative
- The three iteration idioms (range-for, indexed-for, iterator) and when to reach for each
- Defining a `struct`, accessing its fields with `.`, putting structs in a vector
- The `const auto&` pattern in a range-for
- The `break` statement and when it is the right tool

3 `std::vector`, *range-for*, and your first *struct*

- Structured bindings (`auto& [a, b] = pair`) at sight-recognition level
- The three 2D representations and how to choose between them

If `grade_v3` through `grade_v5` make sense end to end — including *why* the 100-input bug exists in `v3` and `v4` and *why* `v5` does not have it — you are ready for chapter 4.